

Deep learning for financial time series

CQF Final project

By Anna Grebentchikova

Deep learning tries

Stock market prediction game

Uncertainty reigns.

Table of Contents

Introduction	3
Methodology.....	4
Ideation	4
Target of analysis: the CUKX	4
Architecture of solution, including model & computations	4
Label: positive trend	6
Model: the Long Short-Term Memory Model – a type of Recurrent Neural Network.....	6
Data Collection & Feature Engineering.....	8
Exploratory Data Analysis	9
Descriptive Statistics	9
Plot CUKX	10
Cleaning & Imputation	11
Additional features & target creation.....	11
Additional feature creation.....	11
Target creation.....	11
Feature Selection & Scaling	12
Feature Selection with Boruta	12
Confirmed features vs Confirmed & Tentative features.....	13
Further feature selection	15
Feature Scaling.....	15
Model Building & Optimization	16
Model building	16
Model optimization.....	19
Hyperparameter tuning	20
Results & Discussion	21
Model Evaluation & Backtesting.....	21
Performance on Training vs Test data	21
Confusion Matrix.....	21
Classification Report	22
Receiver Operator Characteristic Curve (ROC) and Area under ROC (AuC)	23
Conclusion on the basis of Confusion Matrix, Classification Report and ROC.....	23
Back testing a buy-and-hold strategy based on our model	24
Conclusion.....	26
References	27

Introduction

Deep learning is a subset of machine learning that uses artificial neural networks with multiple layers to analyse and make predictions based on large amounts of data. These neural networks are designed to simulate the way the human brain processes information, allowing them to identify complex patterns and make decisions. Deep learning is used in a wide range of applications, including image and speech recognition, natural language processing, and computer vision. It has also been used in finance, and other industries to analyse large amounts of data and make predictions.

Whereas the predictions of deep learning in the fields of image- and speech recognition, and NLP are tremendously faithful, they are less successful in prediction financial time series (such as the stock market). There are several reasons for this:

1. Data complexity: Financial time series data is often noisy, non-linear, and highly complex. It can be affected by a wide range of factors, such as economic indicators, political events, and market sentiment. This makes it challenging for deep learning models to identify patterns and make accurate predictions.
2. Limited data availability: Financial data is often limited, especially for historical data, which can make it difficult to train deep learning models.
3. Non-stationarity: Financial time series data is often non-stationary, meaning that the underlying statistical properties of the data change over time. This can make it difficult for deep learning models to generalize to new data.
4. Overfitting: Deep learning models can be prone to overfitting, especially when trained on limited data. This can lead to models that perform well on the training data but poorly on unseen data.

Still, literature review shows that applying deep learning techniques to predicting financial time series can be tremendously fruitful. For instance, a study by Kim et al. (2018) used a long short-term memory (LSTM) network combined with a convolutional neural network (CNN) to predict stock prices. They managed to create a SP500 ETF predictor with which a profit of 17% could be generated – 14% higher than a buy and hold strategy over the same time period. Such a strong result is not unique: Shen and Shafiq (2020) combined random forest techniques (RF), principal component analysis (PCA) and LSTM to produce a model with an accuracy¹ of 0.93, a precision² of 0.96 and a recall³ of 0.96 for Chinese stock market predictions.

This literature suggests that deep learning models can be effective in predicting stock market movements and identifying profitable trading opportunities. In this project, we will attempt to do the same for the FTSE100. Specifically, we want to predict upward price movements.

¹ Accuracy: measure of the performance of a classifier. The meaning of this, and that of precision and recall, will be further explained in the model optimization sub-chapter

² Precision: measure of a classifier's ability to correctly identify positive instances among all instances it predicts as positive.

³ Recall: measure of a classifier's ability to correctly identify all relevant instances, regardless of whether they are predicted as positive or negative.

Methodology

In this project, the **7 steps to model building** framework shall be followed to create a deep learning model for predicting the direction of FTSE 100 returns.

Step	Workflow
1	Ideation
2	Data Collection & Feature Engineering
3	Exploratory Data Analysis
4	Cleaning & Imputation
5	Feature Selection & Scaling
6	Model Building and Optimization
7	Model Evaluation & Back testing

Table 1: overview of the 7 steps of model building

Ideation

Target of analysis: the CUKX

As subject of study the ETF iShares Core FTSE 100 UCITS ETF GBP (ACC) - the CUKX - has been chosen. This ETF represents the FTSE100, and is a total return index: it includes dividends.

Reason for choosing the FTSE1000

The FTSE100 was chosen for several reasons. First, after years of underperforming the SP500, the FTSE100 finally is outperforming: whereas the SP500 fell ~19% during 2022, the FTSE gained ~1%. Second, the constituents of the FTSE100 are interesting. While one would expect the British index to be made up of British companies, in fact a large part of constituents are multinationals who make most of their revenue (and profit) overseas. Hence, a weaker pound - as seen over the last years - can even help the FTSE100. Finally, over the last 10 years, the FTSE100 (and the CUKX) have seen both dramatic rises and falls, which makes our analysis interesting.

Reasoning for choosing a total return index

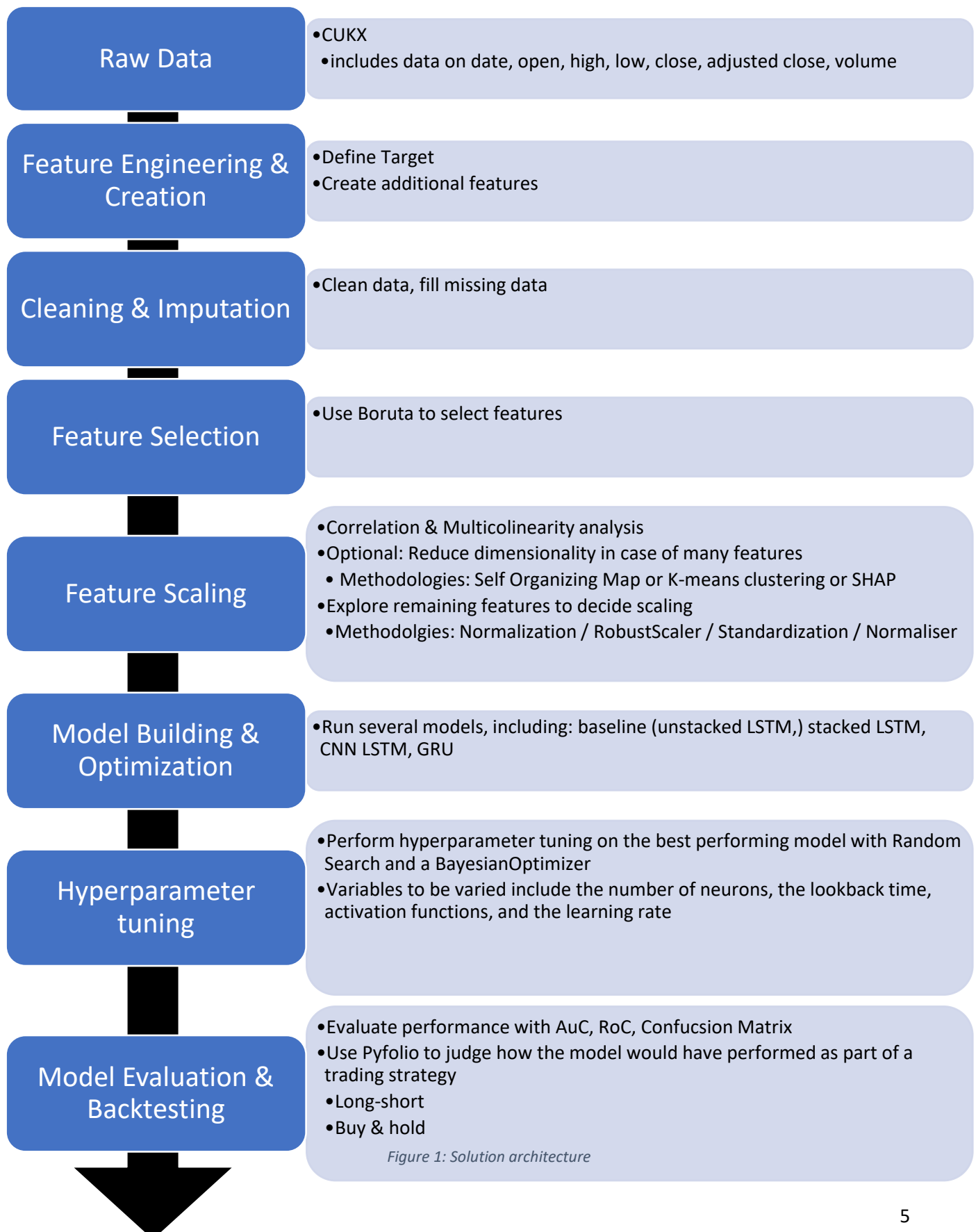
Stock indices come, broadly speaking, in two forms: total return indices (which include dividends) and price indices (which exclude dividend payments). The latter 'drop' after dividend payments take place, to correct for the value extracted from the company's equity capital. ETFs mirror this, as they come in two forms, too: accumulating - with dividends reinvested - and distributing - with dividends paid out to the investor.

For the purpose of our analysis, we have chosen an accumulating index. This saves us from having to worry about falls in equity prices which have little to do with the fundamental economics, and the meaningless noise caused by the (somewhat) random timing of dividend payments.

Architecture of solution, including model & computations

We will use the Long Short-Term Memory (LSTM) model to predict upward price movements of the CUKX. How this model works will be explained further in this section. However, our solution – our pipeline – is broader than simply the LSTM model. Before we can feed our data into the model,

several steps will have to be performed. Schematically, the workflow from data to result looks as follows:



Label: positive trend

Every machine learning algorithm needs a (target) label, which the algorithm learns to predict on the basis of the features. For our label we choose the positive trend: we want to predict 'buy' moments, when we expect the CUKX to go up. Stylized this leads to the following labelling:

$$y = \begin{cases} +1 & \text{if } p_{t+1} - p_t > 0 \\ 0 & \text{otherwise} \end{cases}$$

with p_t representing the price on day t .

However, this above labelling has a slight problem: it will give the 'buy'-signal even if the expected positive trend is negligible. Therefore, we adjust the labelling a bit: we are solely interested in the positive trends that are larger than the 5th decile. This leads us to the following labelling:

$$y = \begin{cases} +1 & \text{if } \frac{p_{t+1} - p_t}{p_t} > D5\left(\frac{p_{t+1} - p_t}{p_t}\right) \\ 0 & \text{otherwise} \end{cases}$$

To go logarithmic, or not to go logarithmic

There are two ways to calculate returns - one can calculate them outright by $r_t = \frac{p_{t+1} - p_t}{p_t}$, or use logarithms. The latter would be in line with the thinking that financial prices seem to be distributed log-normally. However, the sole thing we are doing is put labels (1, 0) on returns based on how they compare to the 5th decile. Both the sign of the return, and its' ranking among returns will remain unchanged by a log-normal transformation. Hence, we opt to stay with the outright calculation of returns.

A note on data leakage

Data leakage happens when information from outside the training dataset is used to create the model. If $D5\left(\frac{p_{t+1} - p_t}{p_t}\right)$ were to be calculated on the basis of the whole dataset, this would constitute data leakage. Hence, it shall solely be calculated on the basis of the training data.

A note on deciles

As a cut off point for the labelling of y , the 5th decile has been chosen. A higher decile is possible, but would cause class imbalance, which is exactly what we tried to avoid by picking the fifth decile.

Model: the Long Short-Term Memory Model – a type of Recurrent Neural Network

The Long Short-Term Memory (LSTM) model is a type of recurrent neural network (RNN) that can handle long-term dependencies - relationships between data points that are separated by a large number of time steps – particularly well. As these types of dependencies play a role in finance, using an LSTM is an obvious solution.

However, before we explain what sets LSTM apart from 'regular' RNN, to understand the general model architecture, it is key to understand how RNN's work. Schematically, RNN's can be said to consist of 3 types of layers: an input layer, hidden/recurrent layers, and an output layer. This overview can be seen below in figure 1. Note that there can be many hidden layers stacked on top of one another.

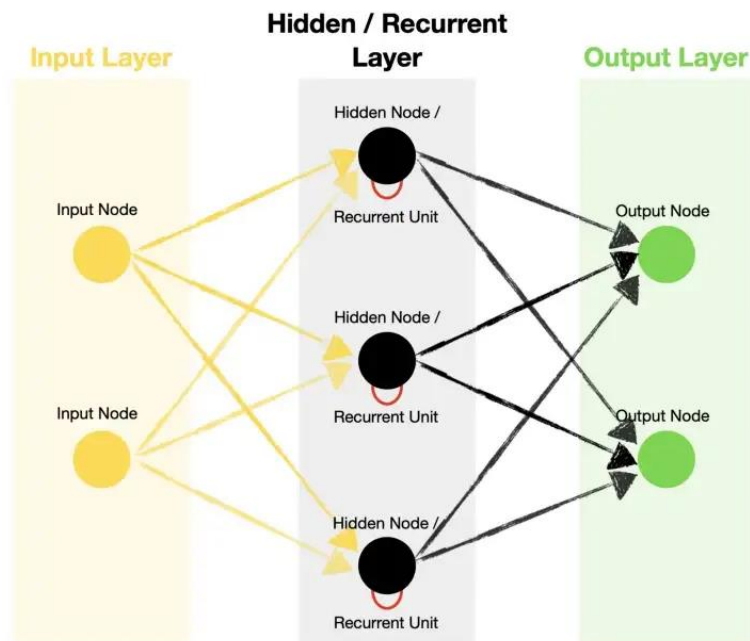


Figure 2: Schematic representation of a RNN

LSTM-networks differ from regular RNN in one key aspect: instead of using 'regular' RNN cells in hidden layers, LST cells are used. These can selectively preserve and propagate information over long sequences, allowing it to learn long-term dependencies in the data. This allows the network to keep track of past information while processing new information, thus making it more efficient at modelling sequential data.

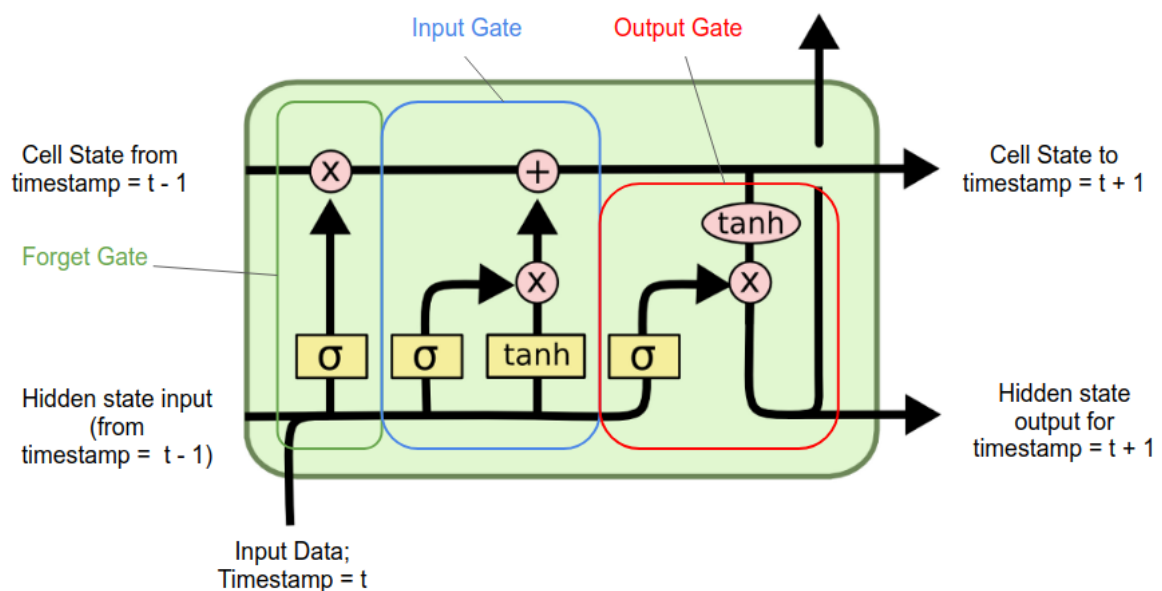


Figure 3: Schematic overview of an LSTM cell, where the $\times$ represents multiplication, $+$ represents addition and σ and $\tanh$ represent activation functions.

An LSTM cell consists of an input gate, a forget gate, and an output gate, which control the flow of information into and out of the cell. The input gate controls what information is added to the cell state, the forget gate controls what information is removed from the cell state, and the output gate controls what information is output from the cell. The exact workings are governed by mathematical equations, as can be seen in figure 3.

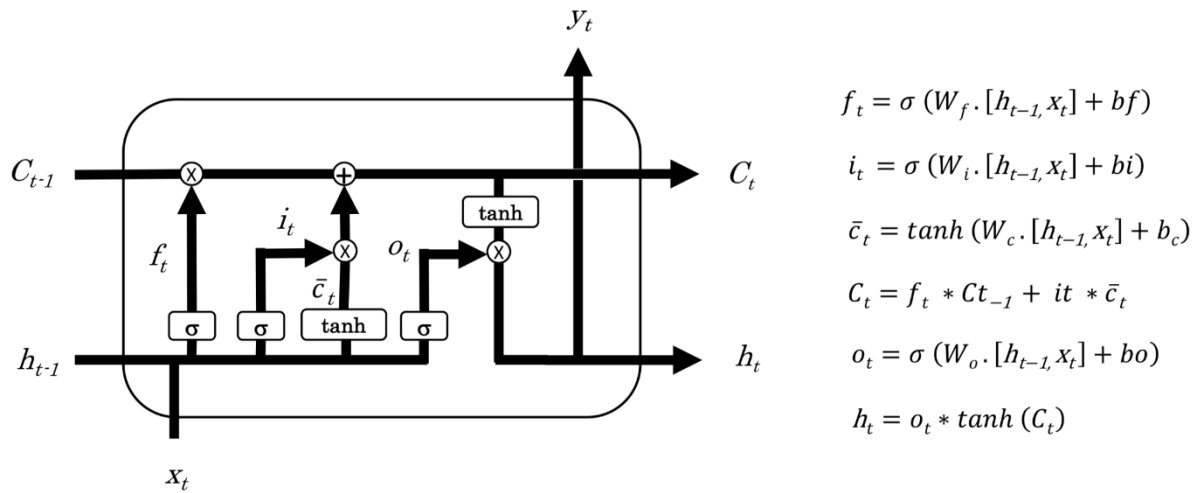


Figure 4: Overview of equations governing the LSTM-cell

Symbol	Meaning
C_t, C_{t-1}	Cell states. Aggregates all data from previous processed time steps, and allows for the long-term memory effects of the LSTM C_{t-1} feeds in from the previous LSTM cell to get updated to C_t
H_t, H_{t-1}	Hidden states. Characterizes the previous time-step (e.g., more recent data). H_{t-1} feeds in from the previous LSTM cell to get updated to H_t
σ	Sigmoid function, serves as activation function and outputs number between 0 & 1
$\tanh$	Tan hyperbolicus, serves as activation function and outputs number between -1 & 1
x_t	Data input at current timestep
b_x	Bias at gate x
W_x	Weights at gate x
$f_t, i_t, \bar{c}_t, o_t$	Forget gate function, input gate function, characterization function and output gate function

Table 2: The equations used by the LSTM cell

Data Collection & Feature Engineering

The sole raw data collected is that of the CUKX from yahoo finance for the last 8 years (since 2015/1/1).

The included features are:

- Open
- High
- Low
- Close
- Adjusted Close

An extra column, Returns, that contains the return from one day to the next is created on the basis of the Adjusted Close data. This column shall be used to create our target.

Additionally, the *pandas_ta* library is used to create 200+ features that stem from technical analysis.

Exploratory Data Analysis

Descriptive Statistics

	count	mean	std	min	25%	50%	75%	max
Open	2025	10940	1406	7827	9820	11038	11962	13814
High	2025	10994	1402	7827	9874	11072	12012	13890
Low	2025	10881	1411	7671	9750	10994	11892	13792
Close	2025	10938	1408	7721	9811	11030	11954	13890
Adj Close	2025	10938	1408	7721	9811	11030	11954	13890
Volume	2025	22591	51819	202	7396	13298	23703	1429589
Returns	2025	0.00029	0.01077	-0.09588	-0.00470	0.00063	0.00551	0.09421

Table 3: descriptive statistics of CUKX data

Date	Adj Close
2015-01-02	8752.5
2015-01-05	8570.5
2023-01-05	13756.0
2023-01-06	13890.0

Table 4: oldest and most recent data in our dataset on CUKX

The above tables - the the statistical summary table combined with oldest and newest CUKX data - show that over the last 8 years, CUKX has moved from ~ 8750 to ~13890, while experiencing a high of ~13890 and a low of ~7720 in between.

The mean is ~10938, with a standard deviation of ~1408: the latest close in our data set is outside of 1 standard deviation.

To gain more insight into CUKX, we plot the Adjusted Close and the histogram of returns below.

Plot CUKX

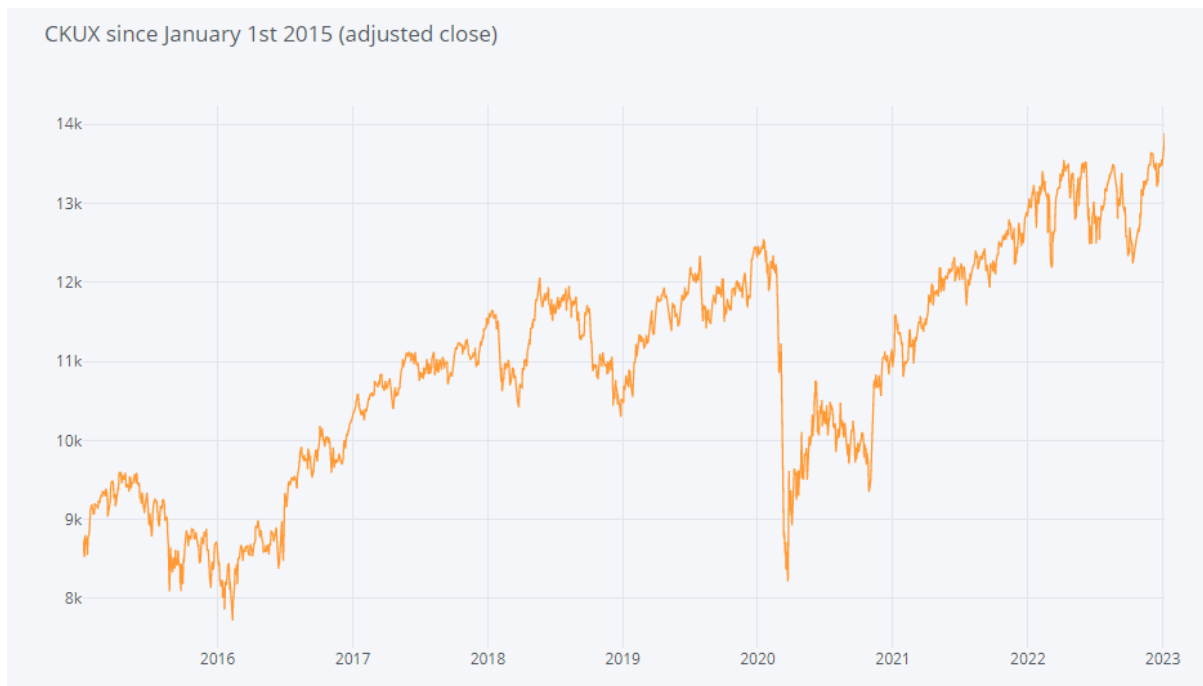


Figure 5: The CUKX returns since 2015 (adjusted close)

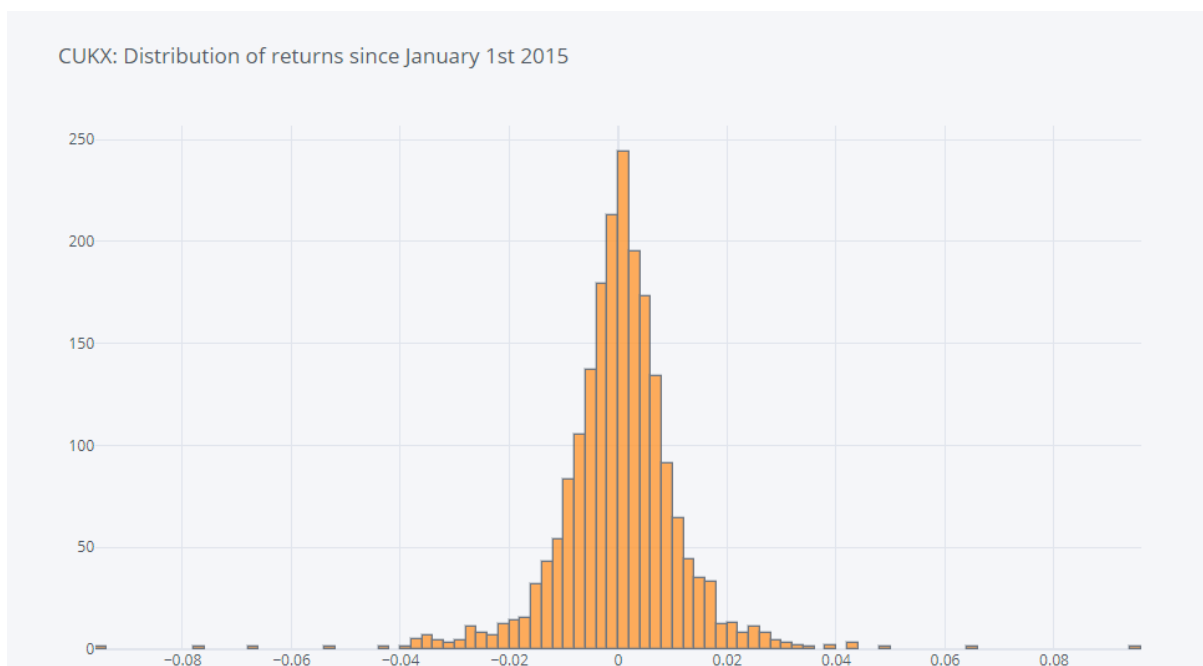


Figure 6: Distribution of CUKX returns since 2015

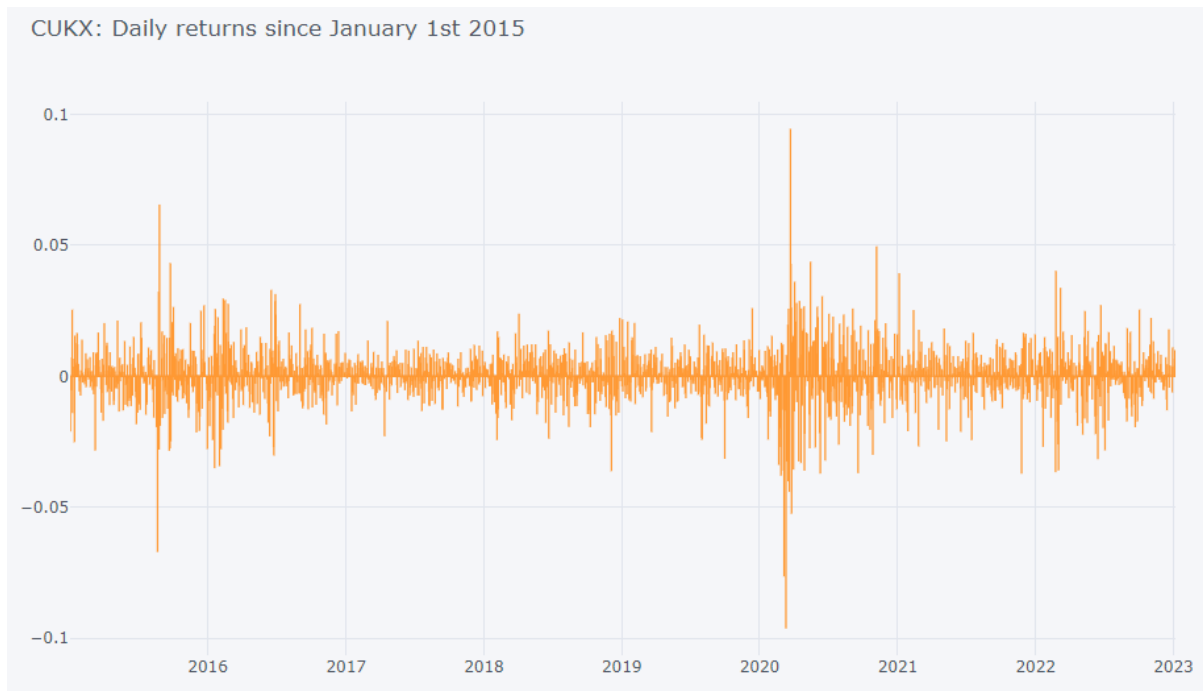


Figure 7: Daily CUKX returns since 2015

Adjusted Close

From the plot of the adjusted close we see the all over the CUKX has trended upward over the last seven years, bar two shocks: around 2016 (Brexit) and 2020 (Covid-19). The latter shock was especially volatile, with a movement well above the 1408 standard deviation in a short time frame. Recently, the CUKX seems to have become more volatile.

Returns

The returns seems to follow a Gaussian distribution, with a peak slightly above 0, and some far outliers in the positive and negative territory. From the daily return bar-graph we can see both those positive and negative returns took place around our shock periods of 2016 and 2020. Later, when scaling the data, we must keep these outliers in mind.

Cleaning & Imputation

A summary table showed no zero values, so we can continue with the data as is.

Additional features & target creation

Additional feature creation

After analysing the CUKX, it is time to use *pandas_ta* to create 200+ technical features. Quick analysis shows that this leads to a significant amount of missing values (10500). Hence, additional data cleaning is required in the form of dropping the first 100 rows, and removing certain features. This leaves 37 missing values which can be solved by backward filling. Afterwards, we once again have 0 missing values.

Target creation

We previously defined our target as

$$y = \begin{cases} +1 & \text{if } \frac{p_{t+1} - p_t}{p_t} > D5\left(\frac{p_{t+1} - p_t}{p_t}\right) \\ 0 & \text{otherwise} \end{cases}$$

but we are yet to calculate the 5th decile of the training data. As our total data set now consists of 1925 rows, 1540 of these will become the training set, and 385 will become the test data set. As a result the 5th decile of $\mathbf{p}_{t+1} - \mathbf{p}_t$ – calculated solely on the basis of the training data to avoid data leakage – equals 0.0004384. This is slightly over 0, which means that the tiniest positive movement still gets a negative label.

Further analysis shows that the labels are very balanced at 971 vs 954, so no class weight correction will be necessary.

Feature Selection & Scaling

Feature Selection with Boruta

The aim of feature selection is to select the input variables which are most helpful in predicting the target. While there are many feature selection algorithms, here we choose to use the wrapper method Boruta. The reasons for doing so are:

1. Boruta is based on the Random Forest algorithm, and can hence handle unscaled input variables well
2. Boruta can take into account multi-variate relationships
3. Boruta follows an all-relevant selection method, which considers all features which are relevant to the outcome, rather than the minimal optimal method

Boruta works by letting each feature compete against a randomized version of itself (the ‘shadow’), and subsequently seeing which features score higher than a pre-agreed threshold. As the algorithm has a significant amount of randomness, it is run multiple times to establish which features are confirmed to be relevant, which are rejected, and which are tentative. These tentative features can subsequently be accepted and used in further analysis, or rejected.

Before we can deploy Boruta, we must train a RandomForest on the dataset. After some hyperparameter tuning, we find using 1000 estimators, and choosing a depth of 3 works well, and provides an accuracy⁴ of 0.63.

The first run of Boruta is not very fruitful: solely 1 confirmed feature is found, and no tentative features. Therefore, we lower the threshold somewhat. The result can be seen below.

Threshold	Confirmed features	Tentative features
100	1	0
99	1	0
97.5	2	2
95	3	4
90	4	4

Table 5: Boruta Threshold settings and resulting confirmed and tentative features

For our further analysis, we will continue with the results of the 95-threshold, for it is both stringent enough, but does produce sufficient features for further analysis.

⁴ Accuracy is a measure of the performance of a classifier. It is calculated as the number of correct predictions made by the classifier divided by the total number of predictions made. This measure shall be further explained in the model optimization sub-chapter

Feature name	Type	Content
DPO_20	Confirmed	Detrended Price Oscillator
WILLR_14	Confirmed	William's Momentum Indicator
OBVe_12	Confirmed	On-balance Volume
ICS_26	Tentative	Ichimoku Indicator
MACDh_12_26	Tentative	Moving Average Convergence Divergence
BEARP_13	Tentative	Bear Power
PVI_1	Tentative	Positive Volume Index

Table 6: Description of confirmed and tentative features

Confirmed features vs Confirmed & Tentative features

To research whether we can best solely use the confirmed features or the confirmed and tentative features, we shall re-run the RandomForest algorithm, create a correlation matrix and calculate the multicollinearity using the Variance Inflation Factor.

RandomForest Accuracy Comparison

	Confirmed features	Confirmed & Tentative features
Accuracy	0.655	0.647

Table 7: Accuracy comparison of RandomForest algorithm between the Confirmed features, and the Confirmed & Tentative features

Using solely the confirmed features leads to a higher accuracy result, but the difference is not large enough to immediately disregard taking the tentative features forward into the analysis.

Correlation Matrix

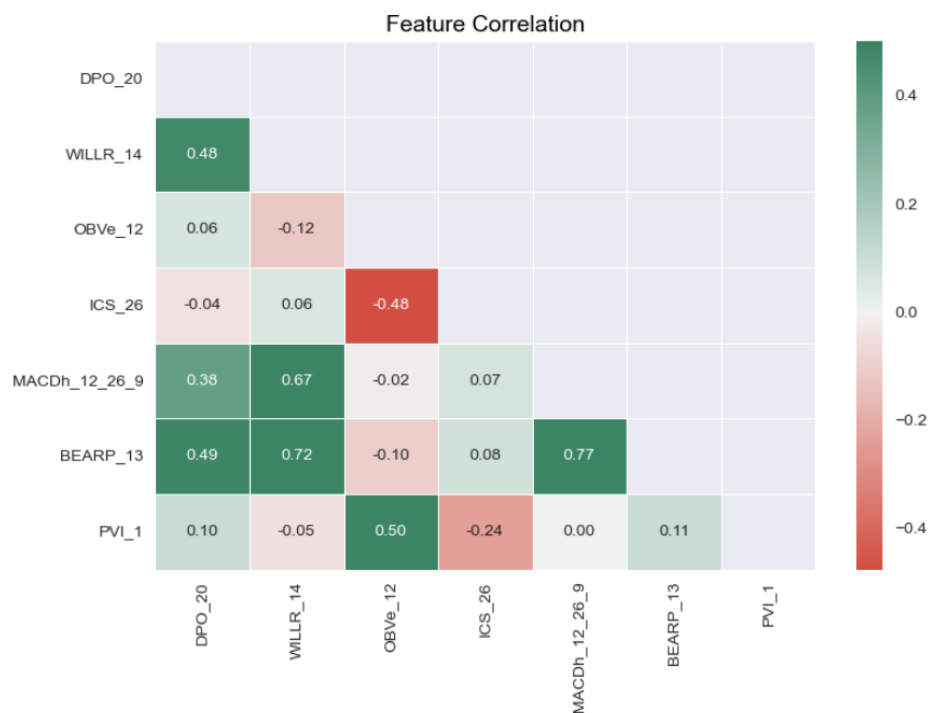


Figure 8: Correlation matrix of confirmed and tentative features

Correlation scores can range between -1 and 1. Generally speaking, correlations are interpreted as follows:

- - 0.25 to 0 and 0 to 0.25: weak correlation
- -0.5 to -0.25 and 0.25 to 0.5: moderate correlation
- -0.75 to -0.5 and 0.5 to 0.75: strong correlation
- -1 to -0.75 and 0.75 to 1: very strong correlation

On the basis of this classification, (very) strong correlations exist between:

- The cluster of BEARP_13, WILLR_14 & MACDh_12_26_9
- PVI_1 and OBVe_12

Furthermore, other features have quite strong correlations as well, although not strong enough to be categorized as 'truly' strong.

Multicollinearity

Multicollinearity occurs when two or more seemingly independent variables (or features) are highly correlated with one another. As a result, the target can just as well be predicted by one variable, as the other. The problem is particularly large when one aims to explain the effect of each feature on the target. Luckily, this will not be the aim of our model and hence multicollinearity is not a significant problem. Still, it is interesting to see if our features have high multicollinearity.

We calculate multicollinearity using the Variance Inflation Factor:

$$VIF = \frac{1}{1 - R^2}$$

Feature	Type	VIF
BEARP_13	Tentative	3.478795
MACDh_12_26_9	Tentative	2.7921051
WILLR_14	Confirmed	2.4178492
OBVe_12	Confirmed	1.7427796
PVI_1	Tentative	1.4509250
DPO_20	Confirmed	1.4112363
ICS_26	Tentative	1.307399

Table 8: VIF table of features

In general, a VIF over 5 is seen as too large. None of our features therefore is disqualified on the basis of VIF.

Conclusion on tentative features

On the basis of the above three analyses, all tentative features except for BEARP_13 should be granted a spot in the feature set, as BEARP_13 showed significant correlation with various other variables – more so than any other feature.

Removing BEARP_13 reduces the accuracy of the model to 0.644.

Further feature selection

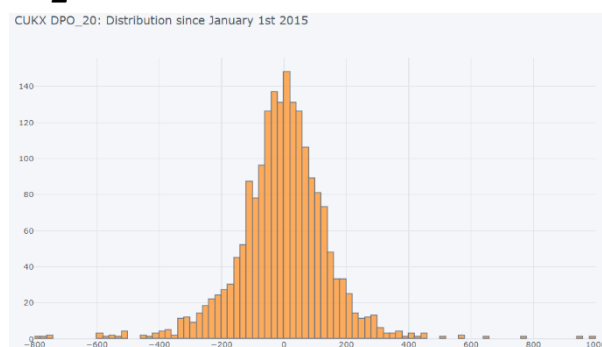
Generally speaking, after performing Boruta, another feature selection algorithm is used, such as SHAP, Lasso, Ridge, or even Chi-Squared or Information Gain. However, Boruta has left us with solely 6 features. Therefore, no further feature selection shall be performed.

Feature Scaling

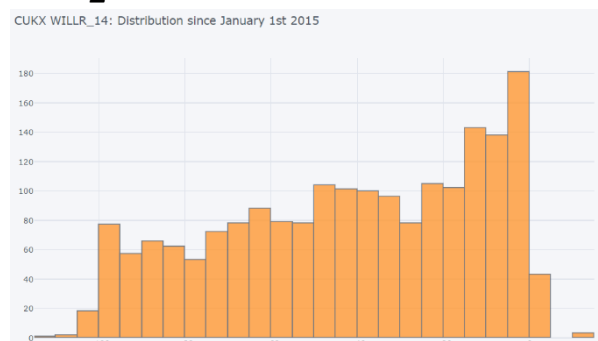
Before the features can be used in an RNN, they must be correctly scaled. To figure out which scaling fits best, data analysis shall be performed.

Distribution of data

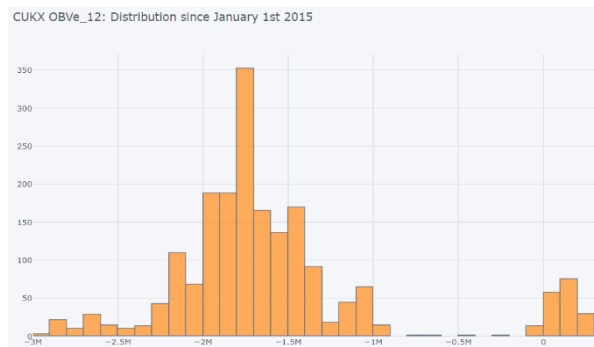
DPO_20



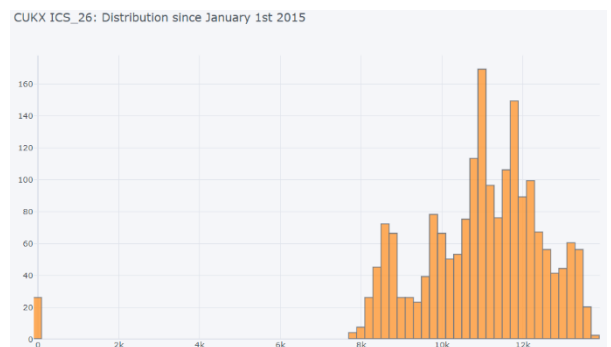
WILLR_14



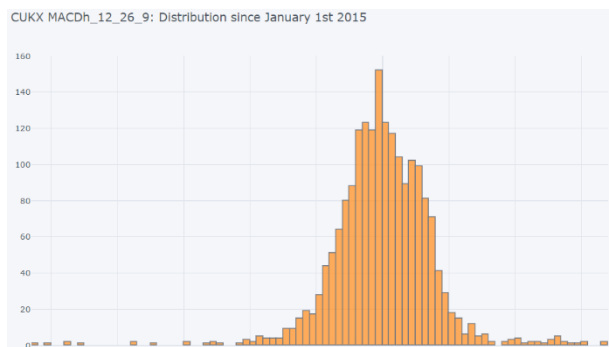
OBVe_12



ICS_26



MACDH_12_26_9



PVI_1

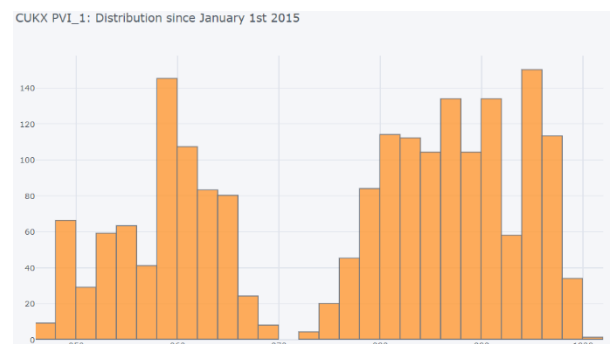


Figure 9-14: Distribution of feature values since 2015

The above figures show one data problem: ICS_26 has a list of zeros. Further analysis shows that this concerns data for the last 26 days before the data-pull cut off of 2023/1/10. Therefore, the last 26 days shall be dropped from the data – both from the features, and the target.

The accuracy after this deleting these days rises to 0.645.

On the basis of data distribution, we shall use the following data scaling:

Feature name	Scaling
DPO_20	StandardScaler
WILLR_14	Normalization
OBVe_12	RobustScaler
ICS_26	Normalization
MACDh_12_26	StandardScaler
PVI_1	Normalization

Table 9: Features and appropriate scaling

Model Building & Optimization

Model building

We can now finally start building our neural network. First, as a baseline, an unstacked LSTM shall be run. Still, there are many hyperparameters to choose from, as can be seen in the table below. The

chosen hyperparameter values stem from what is common in the field when analysing financial time series.

Hyperparameter	What does the hyperparameter do?
Batch Size	Number of samples processed before the model is updated.
Dense layer Activation	Defines whether the output is the result of the previous layer, or whether an additional function is applied.
Dropout Value	Defines whether input and recurrent connections to LSTM units are probabilistically excluded from activation and weight updates while training a network. This has the effect of reducing overfitting and improving model performance.
Early Stopping	Specify an arbitrary large number of training epochs, after which training the model will stop once performance stops improving.
Epochs	How often the neural network will be trained with all the training data per cycle
Initialization	Defines the initial values for the parameters in neural network models prior to training the models on a dataset.
Hidden Layers	Amount of hidden layers in the neural network
Learning Rate	Defines how much the model is changed each time the model weights are updated.
Lookback	Defines the amount of timesteps used to predict the subsequent timestep
LSTM Activation	Defines the activation function used in an LSTM layer.
Neuron	Amount of nodes in hidden neural network layer
Optimizer	Function used to define the best solution/fit of the model per training.

Table 10: Neural Network Hyperparameters and what they do

Hyperparameter	Value options	Chosen Value
Batch Size	32, 64, 128	32
Dense layer Activation	None, Sigmoid, RELU	Sigmoid
Dropout Value	0 – 0.5	None
Early Stopping	No, Yes: 10, Yes: 20	Yes: 10
Epochs	5-500	500
Initialization	Xavier, Random	Xavier
Hidden Layers	1 – 5	1
Learning Rate	0.0001 - 1	0.001
Lookback	1-60	21
LSTM Activation	Sigmoid, Tanh, RELU, ELU	ELU
Neuron	1-1000+	10
Optimizer	Adam, RMSprop	Adam

Table 11: Hyperparameter setting during baseline run

The neural network trains to minimize loss, and the results of each model will be defined in terms of Loss, Accuracy, Precision and Recall.

- **Loss:** represents the degree of error or difference between the predicted output and the actual output of a machine learning model. The goal of training a model is to find the values of the model's parameters that minimize the loss.
- **Accuracy:** measures the performance of a classifier. It is calculated as the number of correct predictions made by the classifier divided by the total number of predictions made.
- **Precision:** measures the classifier's ability to correctly identify positive instances among all instances it predicts as positive. It is calculated as the number of true positives (i.e., the number of correctly predicted positive instances) divided by the number of true positives plus the number of false positives (i.e., the number of instances predicted as positive that are actually negative).
- **Recall:** measures the classifier's ability to correctly identify all relevant instances, regardless of whether they are predicted as positive or negative. It is calculated as the number of true positives (i.e., the number of correctly predicted positive instances) divided by the number of true positives plus the number of false negatives (i.e., the number of instances that are actually positive but predicted as negative).

The results for the baseline model are as follows:

	Baseline: Unstacked LSTM	
	Train	Test
Loss	0.606	0.642
Accuracy	0.655	0.638
Precision	0.645	0.641
Recall	0.687	0.702

Table 12: Baseline LTM model results

The model did not improve upon our RandomForest: our latest accuracy on the test data was 0.645, whereas the neural network has an accuracy of 0.621. Still, the results are better than ‘at random’, and the Precision and Recall scores look good as well.

The data shows slight overfitting – with a training accuracy of ~0.04 higher than that of test data. However, this is not significant enough to introduce a large Dropout Value. In the subsequently trained models, the Dropout Value shall hence be set to 0.3.

Model optimization

	Baseline: Unstacked LSTM		Stacked LSTM (3)		Stacked LSTM (5)		Unstacked GRU		Stacked GRU (3)	
	Train	Test	Train	Test	Train	Test	Train	Test	Train	Test
Loss	0.606	0.642	0.636	0.659	0.611	0.674	0.56	0.687	0.624	0.654
Accuracy	0.655	0.638	0.634	0.604	0.667	0.602	0.68	0.599	0.648	0.637
Precision	0.645	0.641	0.625	0.616	0.661	0.63	0.689	0.626	0.637	0.646
Recall	0.687	0.702	0.67	0.649	0.683	0.58	0.655	0.58	0.683	0.681

Table 13: Overview of Training and Test score metrics for several neural networks

From the above table, it is clear that the Unstacked LSTM and the Stacked GRU outperform while using the base values for the hyperparameters. Comparing these two, the Precision is higher for Stacked GRU, while the Recall of the Unstacked LSTM is better. Interestingly, the RandomForest had higher Accuracy than both.

As a baseline for the hyperparameter tuning we shall use the unstacked LSTM model, as this project pertains the LSTM, and not the GRU model.

Hyperparameter tuning

The hyperparameters that shall be tuned are listed in the table below:

Hyperparameter	Researched Tuning options	Chosen Value Base lien
Dense layer Activation	Sigmoid, RELU	Sigmoid
Learning Rate	0.0001 – 0.1	0.001
Lookback	10-60 ⁵	21
LSTM Activation	Sigmoid, Tanh, RELU, ELU	ELU
Neuron	16-256	256

Table 14: overview of the hyperparameters we will tune for

Two algorithms shall be used for hyperparameter tuning: Random Search and BayesianOptimization. The first excels in is speed, and literature shows that the found accuracy is decently close to that of more advanced algorithms. It works by randomly sampling the parameter space of a model or a function, and evaluating the function at each point. The second is an advanced algorithm that iteratively selects the parameters that maximize the expected improvement. It takes longer, but its results tend to be superior.

Hyperparameter	Optimal per Random Search	Optimal per BayesianOptimization
Dense layer Activation	Sigmoid	Sigmoid
Learning Rate	0.001	0.001
Lookback	50	55
LSTM Activation	Sigmoid	tanh
Neuron	16	128
Resulting Accuracy on Test	0.657	0.657

Table 15: Hyperparameter tuning results

The results of the Random Search and the Bayesian Optimization are exactly the same – down to the comma. Interestingly, the algorithms pick the same Dense layer Activation and Learning Rate, but differ in their lookback, LSTM Activation and Neurons.

⁵ Performing the hyperparameter tuning backward – from the largest to the smallest value leads to significant improvement in accuracy and is therefore implemented in the code.

Results & Discussion

Model Evaluation & Backtesting

The best performing algorithm up to now is an unstacked LSTM, with hyperparameters as defined in the second/third column of table 15. In this chapter we'll evaluate how this algorithm performs in several ways, from getting a sense of over- and underfitting to looking at the Confusion Matrix, the Classification Report and Receiver Operator Characteristic Curve. Finally, a back test will be performed to see how the algorithm would function in the 'real' world when trading.

Performance on Training vs Test data

When trained once more with the optimized hyperparameters, the model shows a training data accuracy of 0.641, and a test data accuracy of 0.635, indicating slight overfitting.

Confusion Matrix

The Confusion matrix shows the performance of the classification model on a set of test data for which the true values are known. It has two axes: the 'true' label: and the 'predicted' label. A perfect model would predict only true labels, but this rarely happens in the real world. Hence, in addition to showing the amount of 'True Positive' and 'True Negative', it also shows 'False Positive' (where the model predicted 1, but the actual value is 0) and 'False Negative' (where the model predicted 0, but the actual value is 1).

Outcome	Position
True Negative	Upper Left
False Negative	Lower Left
False Positive	Upper Right
True Positive	Lower Right

Table 16: overview of a confusion matrix

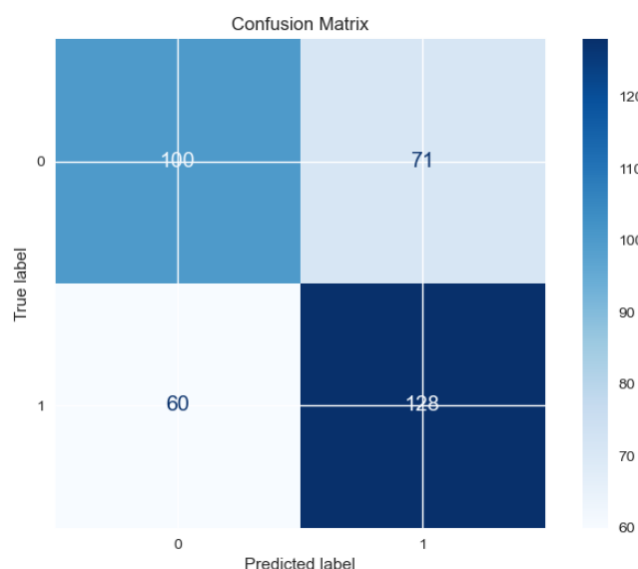


Figure 15: The confusion matrix of the LSTM model

From the confusion matrix one can see that our model is predicts 0/Negative with 62.5% chance of being correct, and 1/Positive with 64.3% of being correct.

Classification Report

The classification report is used to measure the quality of predictions from a classification algorithm.

	precision	recall	f1-score	support
0	0.62	0.58	0.60	171
1	0.64	0.68	0.66	188
accuracy			0.64	359
macro avg	0.63	0.63	0.63	359
weighted avg	0.63	0.64	0.63	359

Figure 16: Classification report of our LSTM model

The above classification report should be read as follows:

Precision: Positive Predictive Value

The precision reflects how good the model is at classifying True Positives and True Negatives. As explained previously, the model scores above average for True Negatives (62%) and True Positives (64%). These scores could be better, but are still impressive: a trading strategy solely needs to be more often correct than incorrect.

Recall: Fraction of Relevant Instances Retrieved

Recall reflects how good the model is at identifying Negatives and Positives. For Negatives the model the model was significantly worse than for Positives at 58% vs 68%. As our model aims to predict the Positive moments, being able to identify close to 7 out of 10 is good.

F1-score: Harmonic Mean of Precision and Recall

The F1 score combines the precision and recall by:

$$\frac{2 * \text{precision} * \text{recall}}{\text{precision} + \text{recall}}$$

Again, we see a better score for the Positive case, than for the Negative case.

Support

The amount of Negatives and Positives in the test data set.

Accuracy: Proportion of Correct Predictions

The accuracy shows how many data points of the test set were correctly labeled, divided by the whole test set. at 64%, the model does well, but could use improving.

Improving the accuracy of the model has been the main focus of this project.

Macro Average

Average of precision/recall/f1-score of different labels.

Weighted Average

Average of precision/recall/f1-score of different labels weighted by the amount of labels per class.

Receiver Operator Characteristic Curve (ROC) and Area under ROC (AuC)

The ROC curve plots the true positive rate (TPR) against the false positive rate (FPR) at various classification thresholds.

Area Under ROC Curve

The area under the ROC curve (AUC) is a measure of how well a model can distinguish between two classes.

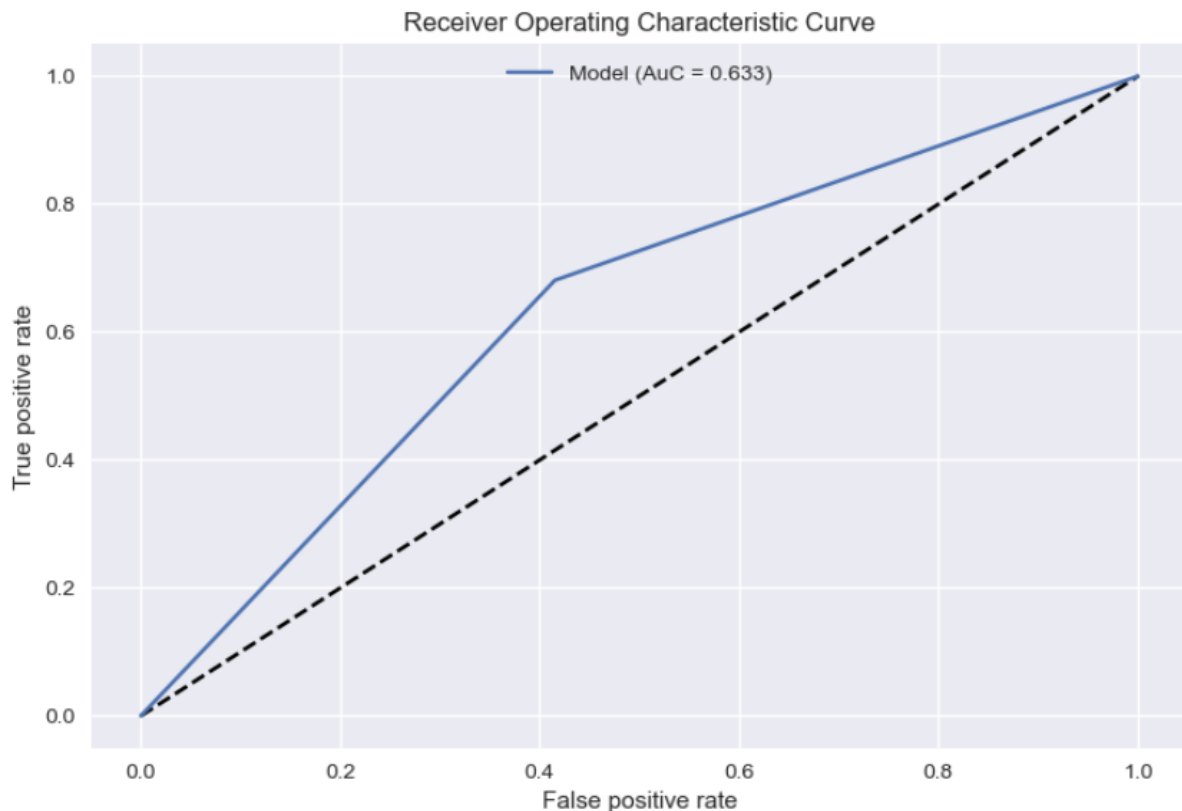


Figure 17: the Receiver Operating Characteristic Curve of our LSTM model

From the plot above one can see that the LSTM algorithm trumps over random predictions. The ROC curve implies that the 'best' score - the greatest distance from the random curve – where the True positive is relatively high (~70%), while the False Positive rate is around 40%.

Conclusion on the basis of Confusion Matrix, Classification Report and ROC

While the model can certainly improve further in identifying Positives, it identifies 68% of them (the recall). On Negative values, our model scores less well.

Back testing a buy-and-hold strategy based on our model

Machine learning models are often used to create trading strategies. As our model excels most at picking up True Positives, it would make sense to create a buy-and-hold strategy on the back of it. In this case, the Positive signals that stem from our LSTM model shall be interpreted as signs to buy the CUKX. It is interesting to see whether one could make a profit by trading on this algorithm.



Figure 17: Back test of our LSTM strategy: first results look good.

The first results of our trading strategy look good. Analysis of several trading metrics shows that the algorithm could indeed be a good basis for a buy-and-hold strategy.

Start date	2021-06-29
End date	2022-11-28
Total months	17

Annual return	57.2%
Cumulative returns	90.4%
Annual volatility	9.3%
Sharpe ratio	4.90
Calmar ratio	25.11
Stability	0.99
Max drawdown	-2.3%
Omega ratio	3.29
Sortino ratio	11.55
Skew	1.31
Kurtosis	4.50
Tail ratio	2.27
Daily value at risk	-1.0%

Figure 18: Analysis of our Buy-and-Hold strategy

With an annual return of 57.2%, the strategy looks very strong, especially as the daily value at risk of -1% is relatively small. The maximum drawdown of -2.3% is an indicator of a strong trading strategy as well. Additionally, The Sharpe ratio of 4.9 indicates the strategy is very strong.

The below figure, which shows the rolling Sharpe ratio, further emphasizes this point: the Sharpe ratio is consistently good.

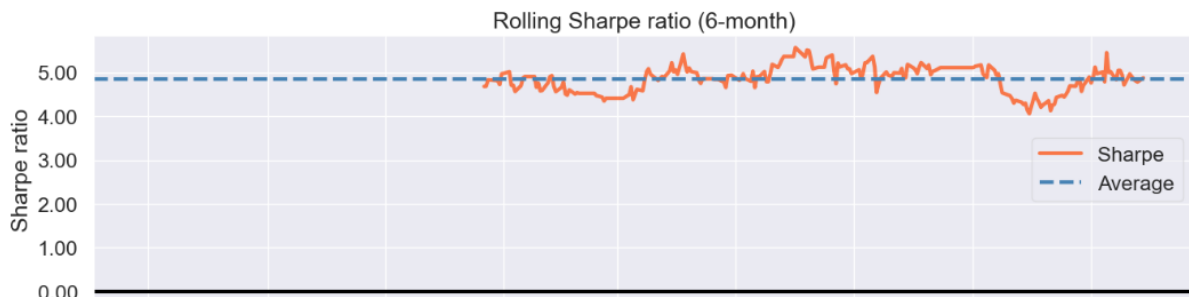


Figure 19: Rolling 6-Month Sharpe Ratio of our buy-and-hold strategy

Given these strong results, it is interesting to see where these result stem from. Hence, the below figure shows the monthly returns.

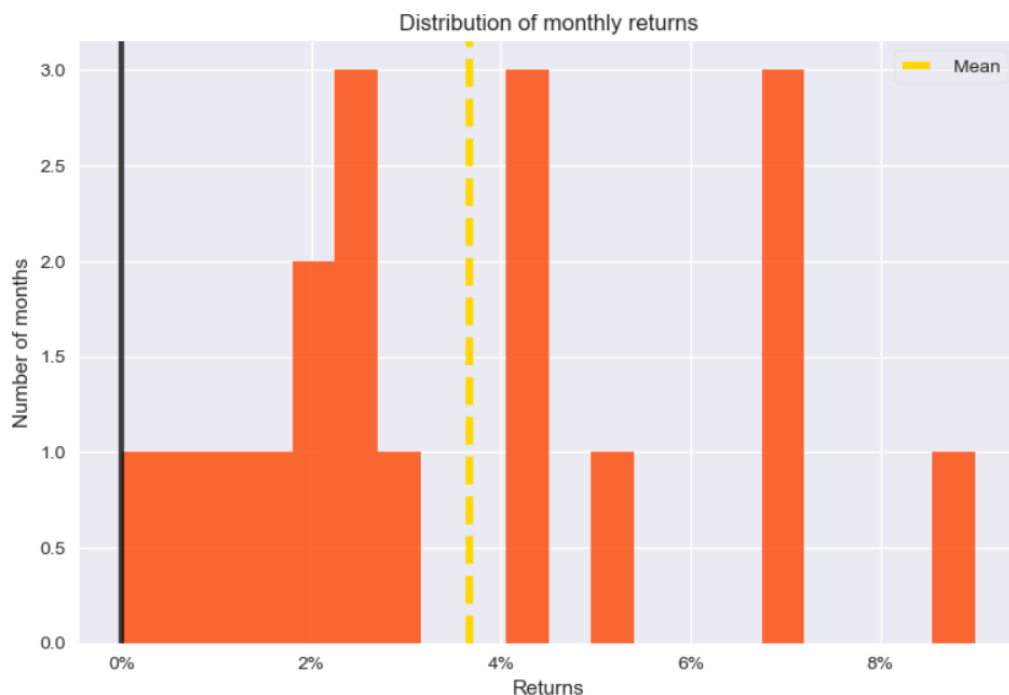


Figure 20: Distribution of monthly returns of our trading strategy

Per the figure above, no month had negative returns, but some months scored very high, with returns of 9% (1 month) and 7% (2 months). Therefore, we can conclude, that the trading strategy seems to generally not lose money, on average create returns around 4% per month, and in some months significantly outperform.

Conclusion

This project aimed to create a model that would pick up on uptrends of the CUKX by using the LSTM framework. After optimizing the model, and tuning the hyperparameters we are left with a model that can predict positive uptrends with 68% reliability: a strong result. Naturally, this model formed a good basis for a buy-and-hold strategy. Back testing shows the model delivers annual returns of 57%. Perhaps this is simply due to being in a constant regime, but still, this is a fruitful result.

For further analysis it would be interesting to look into the performance of the layered GRU-model, which provided similarly strong results. Furthermore, during hyperparameter tuning the model accuracy on the test data was higher than that of the final model. This suggests that further tweaking of the model could potentially create an even stronger predictive value.

References

Kim, T., & Kim, H. Y. (2019). Forecasting stock prices with a feature fusion LSTM-CNN model using different representations of the same data. PLOS ONE, 14(2), e0212320

Shen, J., & Shafiq, M. O. (2020). Short-term stock market price trend prediction using a comprehensive deep learning system. Journal of Big Data, 7(1), 6

CQF Lectures and Python Labs on Machine Learning and Deep Learning, (2022)

Advanced Machine Learning - I & II, Elective by Kannan Singaravelu, (2022)